Smart Real-Time Monitoring & Inspection Platform

Implementation Plan v3 — SIH Problem Statement #26095 (MoSJE / DoSJE)

Change from v2: development now runs on plain Python + SQLite (no Docker, no Postgres, no Redis) to avoid Windows virtualization/BIOS setup pain. The final architecture (Part 2) is unchanged as a *target* — we're just deferring the pieces that need Docker/Redis until later, and building everything else first.

PART 1 — Project Overview

Same as before: a centralized Django web platform for DoSJE to monitor projects/institutes/NGOs in real time — CCTV, random VC, random inspection assignment, geo-tagged reports, anomaly analytics, citizen transparency tools.

What's different in this version: we build and demo it entirely with Django's built-in dev server (`python manage.py runserver`) and a local SQLite file — nothing to install beyond Python itself. Postgres+PostGIS, Redis, Celery, and Docker come back in later once the app itself is working, or whenever your Docker/virtualization setup is sorted.

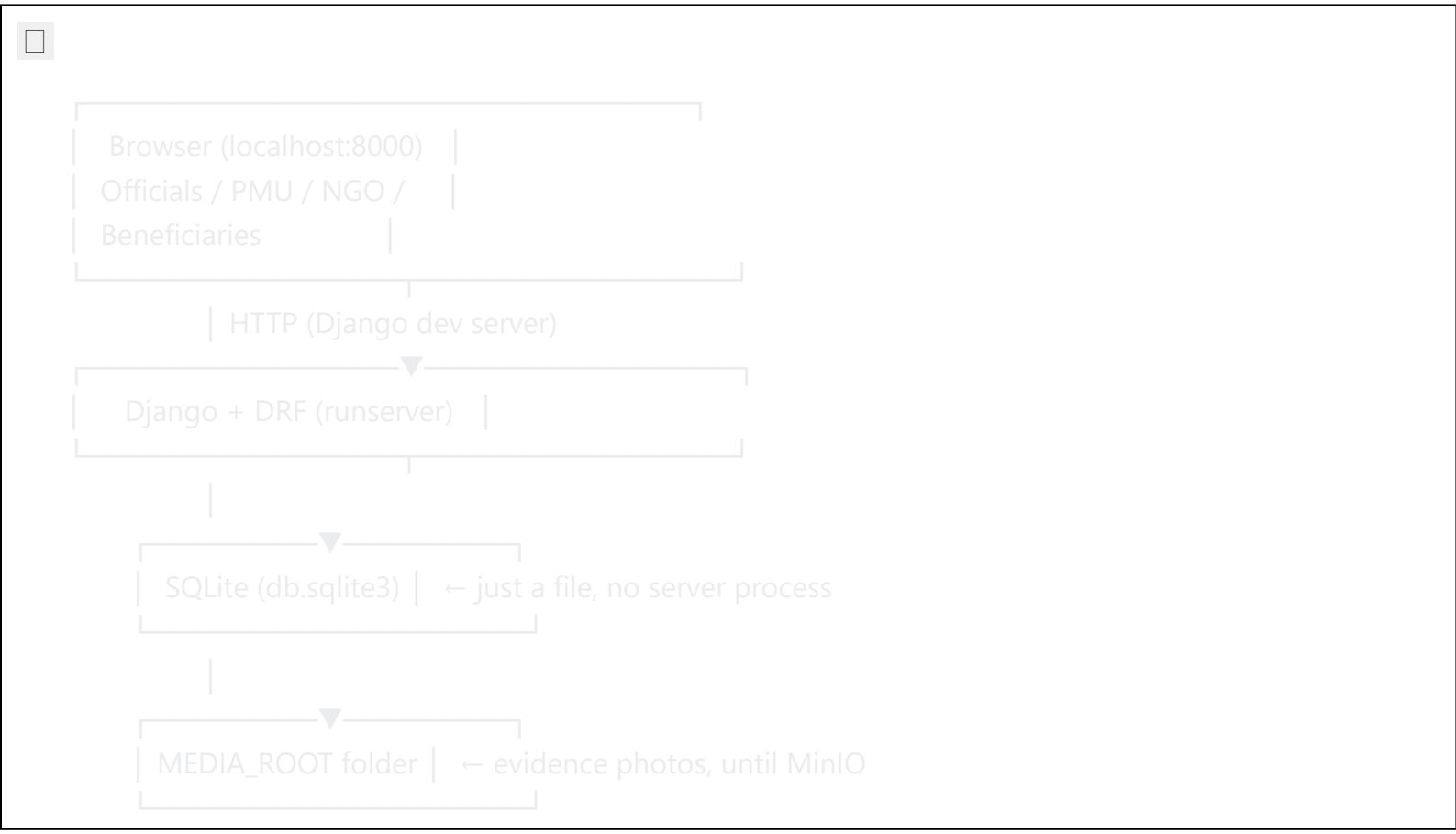
PART 2 — Tech Stack: Now vs. Later

Layer	Now (local dev, no Docker)	Later (matches original plan)
Backend framework	Django + DRF	*(unchanged)*
Frontend	Django Templates + HTMX/Alpine (or React later)	*(unchanged)*
Database	**SQLite** (a single file, zero setup)	Postgres + PostGIS
Geo-tagging	**Plain `latitude`/`longitude` float fields** + Haversine distance check (`apps/core/geo.py`)	PostGIS `PointField` + spatial queries
Real-time dashboard	**Deferred** — build normal page-refresh views first	Django Channels + Redis (WebSockets)
Background/scheduled jobs (random assignment, random VC)	**Deferred** — trigger manually via a management command or admin action first	Celery + Redis (automatic scheduling)
Video conferencing	Deferred	Jitsi Meet (self-hosted)
CCTV ingestion	Deferred	MediaMTX (RTSP → HLS)

Layer	Now (local dev, no Docker)	Later (matches original plan)
File/evidence storage	**Django's local** `MEDIA_ROOT` (just a folder on disk)	MinIO (S3-compatible)
AI/ML	Deferred	FastAPI microservice
Notifications	**Deferred** — Django messages framework / email later	In-app (Channels) + email
Auth	Django session auth	*(unchanged)*
Orchestration	**None needed** — just a Python virtual environment	Docker Compose

Nothing in this table is a wrong turn — it's the same destination, reached in smaller steps. Every "Deferred" item has a note in the code (see `docker-later/` folder and commented-out blocks in `config/settings.py`) showing exactly what to uncomment/install when you're ready for it.

PART 3 — Architecture (Local Dev)



Everything on the right side of the v2 diagram (Redis, Celery, MediaMTX, Jitsi, MinIO, AI microservice) is not running yet — those get added back one at a time in later phases, each behind a Docker Compose block already sitting in `docker-later/docker-compose.yml`.

PART 4 — Core Modules — Status

#	Module	Status
4.1	Auth & RBAC	✅ **Built** — custom `User` model, 8 roles, in `/admin/`
4.2	Project/Institute/NGO Registry	✅ **Built** — Scheme/NGO/Institute/Project/Staff/Beneficiary models + API
4.3	Live CCTV Feed Integration	Deferred (needs MediaMTX/Docker)
4.4	Random Video Conferencing	Deferred (needs Jitsi + a scheduler)
4.5	Real-Time Monitoring Dashboard	Deferred (build a normal dashboard page first; add WebSockets later)
4.6	Web-Based Inspection Module	**Next up** — doesn't need Docker at all, pure Django + browser APIs
4.7	Random Inspection Assignment Engine	Buildable now as a **management command** (run manually), automate with Celery later
4.8	Geo-Tagged Reports & Evidence	**Partially built** — `apps/core/geo.py` haversine check ready; needs the Inspection models (Part 4.6) to use it
4.9	AI-Based Anomaly & Attendance Analytics	Deferred (needs the AI microservice)
4.10	Notifications & Alerts	Start with Django's built-in messages framework + console email backend; upgrade later

PART 5 — Additional Features — still all planned, unchanged from v2

No change to the feature list itself (public transparency portal, grievance module, fund tracking, escalation/SLA, multi-language, QR verification, audit log, PDF/Excel reports, document repository, chatbot) — only *when* they get built shifts, since some depend on modules above that are deferred. Buildable without Docker: grievances, audit log (`django-simple-history` — already installed), PDF/Excel export, document repository, multi-language. Needs later infra: nothing in this list strictly needs Redis/Postgres, so most of Part 5 can actually be pulled forward if you want quick wins.

PART 6 — Project Structure (current)

```
Smart-Monitoring-App/
├── config/                 # settings (SQLite active, Postgres/Redis/Celery commented out)
├── apps/
│   ├── accounts/           # ✅ built
│   ├── registry/           # ✅ built
│   ├── core/               # ✅ geo.py helper built; more shared utils as needed
│   ├── inspections/        # ← Phase 2, next
│   ├── assignment_engine/  # ← Phase 3
│   ├── analytics/          # ← later
│   ├── grievances/ / finance/ / documents/ / transparency/   # ← Part 5, pullable forward
├── docker-later/           # Dockerfile + docker-compose.yml, for when Docker is set up
├── manage.py
├── requirements.txt        # pure-Python deps only; Postgres/Channels/Celery commented at bottom
└── README.md               # step-by-step local (venv) setup
```

PART 7 — Roadmap (Local-first, revised)

Phase 0–1 — ✅ Done

Django project skeleton, custom User + roles, Registry app (Scheme/NGO/ Institute/Project/Staff/Beneficiary), running locally on SQLite via a Python virtual environment.

Phase 2 — Inspection Module (next)

`InspectionTemplate`/`InspectionField` (dynamic checklist, editable from `/admin/`)
`InspectionAssignment` + `InspectionReport` + `Evidence` models
Web form for officers: fill checklist, upload photo (`<input type="file" capture>`), capture GPS via the browser's Geolocation API, submit
Server-side geofence check using `apps/core/geo.py` (already built)
PDF report generation (WeasyPrint) — pure Python, no Docker needed

Phase 3 — Random Inspection Assignment Engine

Weighted-lottery algorithm (days since last inspection, risk score, complaint count)
Runs as a **Django management command** you trigger manually (`python manage.py assign_inspections`) — same algorithm and audit-log design as v2, just triggered by hand instead of Celery-scheduled, until Redis is available

Phase 4 — Bring back Docker infra (whenever ready)

Postgres + PostGIS (swap SQLite → real geo queries)
Redis + Celery (automate the Phase 3 command on a schedule)
Django Channels (turn the dashboard into a live-updating one)
MediaMTX (CCTV) + Jitsi (VC)
MinIO (swap local `MEDIA_ROOT` → S3-compatible storage)

Phase 5 — AI Analytics

FastAPI microservice for anomaly detection / attendance analytics

Phase 6 — Remaining Part 5 features

Whichever weren't pulled forward already (public portal, grievances, etc.)

PART 8 — Local Dev Setup (see README.md for the full copy-paste version)

```bash
python -m venv venv
source venv/Scripts/activate      # Git Bash on Windows
pip install -r requirements.txt
cp .env.example .env
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Then visit `http://127.0.0.1:8000/admin/`.

PART 9 — Why deferring Docker doesn't hurt the pitch

Judges care about **working software and a clear story**, not which database file format you used during development.

The architecture story is unchanged: "SQLite → Postgres+PostGIS" and "manual command → Celery-scheduled" are one-line config changes, not redesigns — worth saying explicitly in your presentation as evidence of clean architecture (settings/config isolated from business logic).

Every deferred piece (Redis, Docker, Celery) is *additive infrastructure*, not a missing feature — the actual DoSJE-specific logic (random weighted assignment, geofence validation, audit trail) is real code either way.

Next step: Phase 2 — the Inspection Module. We'll build it in small pieces (models → admin → one web form → geofence check → PDF export), confirming each piece works before moving to the next, the same way we did Phase 0–1.